SUBHAPREET PATRO
2211CS010547(S1-76)

SQT ASSIGNMENT-3

Q) Explain automation testing types with examples? Explain E-Commerce test cases with suitable examples?

A)

Types of Automation Testing

1. **Unit Testing**
    - The most granular level of testing, where individual units of code or components are tested in isolation to verify their functionality and behavior.
    - **Example:** In Java, **JUnit** is a popular framework used for unit testing.
2. **Integration Testing**
    - Focuses on testing how different modules or components interact with each other. It verifies that the interfaces and data flow between modules work as expected.
    - **Example: Selenium WebDriver** can be used for integration testing of web applications.
3. **Smoke Testing**
    - Also known as **Build Verification Testing (BVT)**, it ensures that the critical functionalities of the application work after every code change or release. If smoke tests pass, more detailed testing can proceed.
    - **Example: Selenium WebDriver** can be used for smoke testing in web applications.
4. **Performance Testing**
    - Evaluates the system's **response time, scalability, stability, and resource usage** under different load conditions. Its goal is to identify bottlenecks and ensure the system can handle expected user loads.
    - **Example: Apache JMeter** is widely used for performance testing.
5. **Regression Testing**
    - Conducted after code changes or new releases to ensure that existing functionality still works correctly and that no previously fixed issues have reappeared.
    - **Example: Selenium WebDriver** is commonly used for regression testing of web applications.
6. **Security Testing**

- Identifies vulnerabilities such as **SQL injection, cross-site scripting (XSS),** and other threats to confirm the system protects sensitive data and meets security standards.
- **Example: OWASP ZAP (Zed Attack Proxy)** is a popular tool for security testing.

7. **Acceptance Testing**
   - Performed by **end users or customers** to check whether the application meets business requirements and is ready for production. It is usually based on acceptance criteria defined in user stories or requirement documents.
   - **Example: TestComplete** can be used for acceptance testing of web, desktop, and mobile applications.
8. **API Testing**
   - Validates **Application Programming Interfaces (APIs)** to ensure they return correct responses and provide expected functionality. It verifies communication between different systems.
   - **Example: Postman** is a widely used tool for API testing.
9. **UI Testing**
   - Focuses on the **look, feel, and usability** of an application's user interface to ensure a smooth and intuitive user experience.
   - **Example: Selenium WebDriver** can be used for UI testing of web applications.

E-Commerce Automation Test Cases with Code Examples for Flipkart

1. Unit Testing (Add to Cart logic)

- **Goal:** Verify if adding a product correctly updates the cart count.

```
// Example using JUnit
import static org.junit.Assert.*;
import org.junit.Test;

public class CartTest {
   @Test
   public void testAddToCart() {
      Cart cart = new Cart();
      cart.addProduct("iPhone", 1);
```

```
        assertEquals(1, cart.getTotalItems());
    }
}
```

This checks only the **cart logic** in isolation.

2. Integration Testing (Cart + Payment flow)

- **Goal:** Ensure cart updates are reflected in payment module.

```
@Test
public void testCartToPaymentIntegration() {
    Cart cart = new Cart();
    cart.addProduct("Laptop", 1);

    Payment payment = new Payment();
    boolean result = payment.checkout(cart);

    assertTrue(result);
}
```

Tests **two modules working together**.

3. Smoke Testing (Login + Search basic workflow)

- **Goal:** Validate critical functionality.

```
// Example using Selenium WebDriver
WebDriver driver = new ChromeDriver();
driver.get("https://www.flipkart.com");

// Login
driver.findElement(By.id("login-username")).sendKeys("testuser");
driver.findElement(By.id("login-password")).sendKeys("password");
driver.findElement(By.id("login-btn")).click();
```

```
// Smoke Test → Search
driver.findElement(By.name("q")).sendKeys("iPhone 15");
driver.findElement(By.name("q")).submit();
assertTrue(driver.getTitle().contains("iPhone 15"));
```

Ensures **login + search works** after every build.

4. Performance Testing (Search API load test)

- **Goal:** Simulate 1000 users searching at once.

```
// JMeter/Postman pseudo-script
GET https://api.flipkart.com/search?q=iPhone15
```

Assertions:

- Response time < 2s
- Status code = 200
- JSON contains "iPhone"

Tests **response time and stability**.

5. Regression Testing (Verify checkout still works after code changes)

```
// Selenium test for Checkout Flow
driver.findElement(By.id("add-to-cart")).click();
driver.findElement(By.id("checkout-btn")).click();
driver.findElement(By.id("payment-method")).click();
driver.findElement(By.id("confirm-payment")).click();

assertTrue(driver.getPageSource().contains("Order Confirmed"));
```

Ensures existing **checkout flow isn't broken** by new updates.

6. Security Testing (SQL Injection attempt in search)

```
// Using OWASP ZAP or Postman
GET https://api.flipkart.com/search?q=' OR '1'='1
```

Assertions:

- Status = 400 (Bad Request) or Sanitized
- Response does not reveal database error

Checks if **search input is vulnerable to SQL injection**.

7. Acceptance Testing (End-to-end order placement)

```
@Test
public void testOrderPlacement() {
    driver.get("https://flipkart.com");

    // Search & Add to Cart
    driver.findElement(By.name("q")).sendKeys("Shoes");
    driver.findElement(By.name("q")).submit();
    driver.findElement(By.cssSelector(".add-to-cart")).click();

    // Checkout
    driver.findElement(By.id("checkout-btn")).click();
    driver.findElement(By.id("payment-method")).click();

    assertTrue(driver.getPageSource().contains("Order Confirmed"));
}
```

Validates **real customer journey**.

8. API Testing (Order API)

- **Goal:** Verify order placement API.

```
// Postman (JavaScript test script)
```

```
pm.test("Order API returns success", function () {
    var jsonData = pm.response.json();
    pm.expect(pm.response.code).to.eql(200);
    pm.expect(jsonData.status).to.eql("ORDER_PLACED");
});
```

Ensures API **returns expected JSON response**.

9. UI Testing (Check search bar visibility)

```
// Selenium UI test
WebElement searchBox = driver.findElement(By.name("q"));
assertTrue(searchBox.isDisplayed());
```

Confirms **UI element is present and usable**.